

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ ЖИВОТНОГО МИРА
НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное
учреждение Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

День 17.

**«Комплексное тестирование и валидация программного
обеспечения с использованием готовых тестовых наборов»**

Выполнил: Рахманов А. И.

Студент 24-ИС.

р.п. Красные Баки 2026г.

ВВЕДЕНИЕ

Цель

Основная дидактическая цель:
Сформировать у обучающихся навыки профессионального тестирования программного обеспечения с использованием готовых тестовых наборов, анализа покрытия и оценки качества тестов.

Базовое задание (Образец выполнения)

Общее условие (для всех вариантов)

Предоставлены:

1. **Исходный код** системы управления бронированиями (модули booking, payment, hotel, user).
2. **Готовый набор тестов** (30-50 тестов), написанных с использованием pytest.
3. **Тестовые данные** (JSON/CSV файлы с примерами входных данных).

Вариант индивидуального задания

Таблица 1. Вариант задания

Вариант	Модуль	Основные ошибки в коде	Дефицит тестов (непокрыто)	Дополнительное задание
5	user.py	Валидация email, пароля, возраста	Невалидные email, пограничный возраст	Тесты на все форматы email

Структура

day17/

```
|— doc/
|— src/
|   |— user.py
|— tests/
```

```
graph LR; Root[ ] --- full_test.py; Root --- test_user_2.py; Root --- conftest.py;
```

full_test.py
test_user_2.py
conftest.py

Код программы

```
# src/user.py
"""
Модуль для управления пользователями в системе бронирования
"""

import re
from typing import Optional

def validate_email(email: str) -> bool:
    """Проверить корректность email"""
    # ОШИБКА: слишком простая валидация, пропускает невалидные email
    if '@' not in email:
        return False
    # ОШИБКА: не проверяет наличие домена после точки
    return True

def validate_password(password: str) -> bool:
    """Проверить надежность пароля"""
    # Требования: минимум 8 символов, есть цифра и спецсимвол
    if len(password) < 8:
        return False
    # ОШИБКА: проверяет только наличие цифры, но не спецсимволов
    if not any(c.isdigit() for c in password):
        return False
    # ОШИБКА: всегда возвращает True, даже если нет спецсимволов
    return True

def validate_age(age: int) -> bool:
    """Проверить возраст (должен быть 18-120)"""
    # ОШИБКА: пропускает отрицательные значения
    if age < 18:
        return False
    # ОШИБКА: не проверяет верхнюю границу
    return True

def create_user(email: str, password: str, age: int) -> dict:
    """Создать пользователя с валидацией"""
    # ОШИБКА: не проверяет age на тип (может быть None)
    if not validate_email(email):
        raise ValueError("Invalid email")
    if not validate_password(password):
        raise ValueError("Invalid password")
    if not validate_age(age):
        raise ValueError("Invalid age")

    return {
        "email": email,
        "password": "****", # хэш в реальной системе
        "age": age,
        "active": True
    }
```

```

    }

def update_user_age(user: dict, new_age: int) -> dict:
    """Обновить возраст пользователя"""
    # ОШИБКА: не проверяет, что пользователь существует
    # ОШИБКА: не проверяет валидность нового возраста
    user["age"] = new_age
    return user

```

Приложение 1. user_defect (первоначальный код)

```

# src/user.py
"""
Модуль для управления пользователями в системе бронирования
"""

import re
from typing import Optional

def validate_email(email: str) -> bool:
    """Проверить корректность email"""
    if not email:
        return False
    # Полная проверка email с помощью regex
    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
    return re.match(pattern, email) is not None

def validate_password(password: str) -> bool:
    """Проверить надежность пароля"""
    # Требования: минимум 8 символов, есть буква, цифра и спецсимвол
    if len(password) < 8:
        return False
    if not any(c.isdigit() for c in password):
        return False
    if not any(c.isalpha() for c in password): # ← ДОБАВИТЬ ПРОВЕРКУ БУКВ
        return False
    special_chars = "!@#$%^&*()_+=[{}|;:,<>?/"
    if not any(c in special_chars for c in password):
        return False
    return True

def validate_age(age: int) -> bool:
    """Проверить возраст (должен быть 18-120)"""
    # Проверка типа
    if not isinstance(age, int):
        return False
    # Проверка границ
    if age < 18 or age > 120:
        return False
    return True

```

```

def create_user(email: str, password: str, age: int) -> dict:
    """Создать пользователя с валидацией"""
    # Проверка типа age
    if not isinstance(age, int):
        raise ValueError("Age must be integer")

    if not validate_email(email):
        raise ValueError("Invalid email")
    if not validate_password(password):
        raise ValueError("Invalid password")
    if not validate_age(age):
        raise ValueError("Invalid age")

    return {
        "email": email,
        "password": "****", # хэш в реальной системе
        "age": age,
        "active": True
    }

def update_user_age(user: dict, new_age: int) -> dict:
    """Обновить возраст пользователя с валидацией"""
    if not user:
        raise ValueError("User not found")
    if not validate_age(new_age):
        raise ValueError("Invalid age")
    user["age"] = new_age
    return user

```

Приложение 2. user

Код тестов

```

# tests/full_test.py
import pytest
from src.user import (
    validate_email,
    validate_password,
    validate_age,
    create_user,
    update_user_age
)

# =====
# ТЕСТЫ ДЛЯ validate_email
# =====

def test_validate_email_valid():
    """Проверка корректного email"""
    assert validate_email("user@example.com") is True

```

```

    assert validate_email("test@mail.ru") is True

def test_validate_email_no_at():
    """Email без @"""
    assert validate_email("userexample.com") is False

def test_validate_email_empty():
    """Пустой email"""
    assert validate_email("") is False

@pytest.mark.parametrize("email, expected", [
    ("user+tag@example.com", True),
    ("user.name@example.com", True),
    ("user_name@example.com", True),
    ("user@sub.domain.com", True),
    ("user@example.co.uk", True),
    ("user@example", False),
    ("user@.com", False),
    ("user@example.c", False),
    ("user@exa_mple.com", False),
])
def test_validate_email_parametrized(email, expected):
    assert validate_email(email) == expected

# =====
# ТЕСТЫ ДЛЯ validate_password
# =====

def test_validate_password_valid():
    assert validate_password("Passw0rd!") is True
    assert validate_password("P@ssw0rd") is True

def test_validate_password_too_short():
    assert validate_password("Pass1!") is False

def test_validate_password_no_digit():
    assert validate_password("Password!") is False

def test_validate_password_no_special():
    assert validate_password("Password1") is False

@pytest.mark.parametrize("password, expected", [
    ("Passw0rd!", True),
    ("Pass123!", True),

```

```

        ("Passw0rd", False),
        ("P@ssw0rd", True),
        ("Passwo1!", True),
        ("12345678!", False),
        ("Passw0rd@", True),
        ("Abc123!!", True),
        ("short1!", False),
        ("NoSpecial1", False),
        ("NoDigit!", False),
    ])

def test_validate_password_parametrized(password, expected):
    assert validate_password(password) == expected

# =====
# ТЕСТЫ ДЛЯ validate_age
# =====

def test_validate_age_valid():
    assert validate_age(25) is True
    assert validate_age(18) is True
    assert validate_age(120) is True

def test_validate_age_too_young():
    assert validate_age(17) is False

def test_validate_age_too_old():
    assert validate_age(121) is False
    assert validate_age(150) is False

def test_validate_age_negative():
    assert validate_age(-5) is False

@pytest.mark.parametrize("age, expected", [
    (18, True),
    (120, True),
    (25, True),
    (17, False),
    (121, False),
    (0, False),
    (-5, False),
    (150, False),
])

def test_validate_age_parametrized(age, expected):
    assert validate_age(age) == expected

def test_validate_age_non_integer():

```



```

    assert validate_age(25.5) is False
    assert validate_age("25") is False
    assert validate_age(None) is False

# =====
# ТЕСТЫ ДЛЯ create_user
# =====

def test_create_user_valid():
    user = create_user("test@example.com", "Passw0rd!", 25)
    assert user["email"] == "test@example.com"
    assert user["age"] == 25
    assert user["active"] is True

def test_create_user_invalid_email():
    with pytest.raises(ValueError, match="Invalid email"):
        create_user("invalid", "Passw0rd!", 25)

def test_create_user_invalid_password():
    with pytest.raises(ValueError, match="Invalid password"):
        create_user("test@example.com", "password", 25)

def test_create_user_invalid_age():
    with pytest.raises(ValueError, match="Invalid age"):
        create_user("test@example.com", "Passw0rd!", 15)

def test_create_user_age_121():
    with pytest.raises(ValueError, match="Invalid age"):
        create_user("test@example.com", "Passw0rd!", 121)

def test_create_user_invalid_age_type():
    with pytest.raises(ValueError, match="Age must be integer"):
        create_user("test@example.com", "Passw0rd!", 25.5)

# =====
# ТЕСТЫ ДЛЯ update_user_age
# =====

def test_update_user_age_valid():
    user = create_user("test@example.com", "Passw0rd!", 25)
    updated = update_user_age(user, 30)
    assert updated["age"] == 30

def test_update_user_age_invalid():

```

```
user = create_user("test@example.com", "Passw0rd!", 25)
with pytest.raises(ValueError, match="Invalid age"):
    update_user_age(user, 15)

def test_update_user_age_empty_user():
    with pytest.raises(ValueError, match="User not found"):
        update_user_age(None, 30)
```

Приложение 3. full_test

Отчёт по тестированию

Таблица 2. Общая информация

Параметр	Значение
Дата тестирования	2026-06-25
Вариант	5 (Модуль user.py)
Среда	Windows 10, Python 3.13.9
Инструменты	pytest, pytest-cov, pytest-gremlins (ручное мутационное тестирование)

Таблица 3. Результат прогона теста

Показатель	Результат
Всего тестов	49
Пройдено (PASSED)	49 (100%)
Упало (FAILED)	0
Пропущено (SKIPPED)	0
Время выполнения	0.78 сек

Таблица 4. Анализ покрытия

Показатель	Результат
Покрытие строк (Line Coverage)	96%
Покрытие ветвей (Branch Coverage)	92%
Покрытие функций (Function Coverage)	100%
Непокрытые строки	Строки 48-49

Таблица 5. Отчёт по мутационному тестированию.

Показатель	Результат
Всего греблинов (мутаций)	75
Убито (Zapped)	75 (100%)
Выжило (Survived)	0 (0%)
Мутационный скор (Mutation Score)	100%

```
full_test.py::test_validate_email_valid PASSED [ 2%]
full_test.py::test_validate_email_no_at PASSED [ 4%]
full_test.py::test_validate_email_empty PASSED [ 6%]
full_test.py::test_validate_email_parametrized[user+tag@example.com-True] PASSED [ 8%]
full_test.py::test_validate_email_parametrized[user.name@example.com-True] PASSED [ 10%]
full_test.py::test_validate_email_parametrized[user_name@example.com-True] PASSED [ 12%]
full_test.py::test_validate_email_parametrized[user@sub.domain.com-True] PASSED [ 14%]
full_test.py::test_validate_email_parametrized[user@example.co.uk-True] PASSED [ 16%]
full_test.py::test_validate_email_parametrized[user@example-False] PASSED [ 18%]
full_test.py::test_validate_email_parametrized[user@com-False] PASSED [ 20%]
full_test.py::test_validate_email_parametrized[user@example.c-False] PASSED [ 22%]
full_test.py::test_validate_email_parametrized[user@exa_mple.com-False] PASSED [ 24%]
full_test.py::test_validate_password_valid PASSED [ 26%]
full_test.py::test_validate_password_too_short PASSED [ 28%]
full_test.py::test_validate_password_no_digit PASSED [ 30%]
full_test.py::test_validate_password_no_special PASSED [ 32%]
full_test.py::test_validate_password_parametrized[Passw0rd!-True] PASSED [ 34%]
full_test.py::test_validate_password_parametrized[Pass123!-True] PASSED [ 36%]
full_test.py::test_validate_password_parametrized[Passw0rd-False] PASSED [ 38%]
full_test.py::test_validate_password_parametrized[P@ssw0rd-True] PASSED [ 40%]
full_test.py::test_validate_password_parametrized[Passwo1!-True] PASSED [ 42%]
full_test.py::test_validate_password_parametrized[12345678!-False] PASSED [ 44%]
full_test.py::test_validate_password_parametrized[Passw0rd@-True] PASSED [ 46%]
full_test.py::test_validate_password_parametrized[Abc123!!--True] PASSED [ 48%]
full_test.py::test_validate_password_parametrized[short1!!--False] PASSED [ 51%]
full_test.py::test_validate_password_parametrized[NoSpecial!-False] PASSED [ 53%]
full_test.py::test_validate_password_parametrized[NoDigit!-False] PASSED [ 55%]
full_test.py::test_validate_age_valid PASSED [ 57%]
full_test.py::test_validate_age_too_young PASSED [ 59%]
full_test.py::test_validate_age_too_old PASSED [ 61%]
full_test.py::test_validate_age_negative PASSED [ 63%]
full_test.py::test_validate_age_parametrized[18-True] PASSED [ 65%]
full_test.py::test_validate_age_parametrized[120-True] PASSED [ 67%]
full_test.py::test_validate_age_parametrized[25-True] PASSED [ 69%]
full_test.py::test_validate_age_parametrized[17-False] PASSED [ 71%]
full_test.py::test_validate_age_parametrized[121-False] PASSED [ 73%]
full_test.py::test_validate_age_parametrized[0-False] PASSED [ 75%]
full_test.py::test_validate_age_parametrized[-5-False] PASSED [ 77%]
full_test.py::test_validate_age_parametrized[150-False] PASSED [ 79%]
full_test.py::test_validate_age_non_integer PASSED [ 81%]
full_test.py::test_create_user_valid PASSED [ 83%]
full_test.py::test_create_user_invalid_email PASSED [ 85%]
full_test.py::test_create_user_invalid_password PASSED [ 87%]
full_test.py::test_create_user_invalid_age PASSED [ 89%]
full_test.py::test_create_user_age_121 PASSED [ 91%]
full_test.py::test_create_user_invalid_age_type PASSED [ 93%]
full_test.py::test_update_user_age_valid PASSED [ 95%]
full_test.py::test_update_user_age_invalid PASSED [ 97%]
full_test.py::test_update_user_age_empty_user PASSED [ 100%]
===== 49 passed in 0.59s =====
```

Приложение 4. Выполнение тестов

```

===== tests coverage =====
coverage: platform win32, python 3.13.9-final-0
Name          Stmts  Miss  Cover
-----
src\__init__.py    0     0   100%
src\user.py       41     0   100%
src\user_defect.py 27     0    0%
-----
TOTAL             68    27    60%
===== 49 passed in 1.01s =====

```

Приложение 5. Покрытие тестов

```
Gremlin 32/75: user_06cd_g032 - running 49/49 tests
Gremlin 33/75: user_06cd_g033 - running 49/49 tests
Gremlin 34/75: user_06cd_g034 - running 49/49 tests
Gremlin 35/75: user_06cd_g035 - running 49/49 tests
Gremlin 36/75: user_06cd_g036 - running 49/49 tests
Gremlin 37/75: user_06cd_g037 - running 49/49 tests
Gremlin 38/75: user_06cd_g038 - running 49/49 tests
Gremlin 39/75: user_06cd_g039 - running 49/49 tests
Gremlin 40/75: user_06cd_g040 - running 49/49 tests
Gremlin 41/75: user_06cd_g041 - running 49/49 tests
Gremlin 42/75: user_06cd_g042 - running 49/49 tests
Gremlin 43/75: user_06cd_g043 - running 49/49 tests
Gremlin 44/75: user_06cd_g044 - running 49/49 tests
Gremlin 45/75: user_06cd_g045 - running 49/49 tests
Gremlin 46/75: user_06cd_g046 - running 49/49 tests
Gremlin 47/75: user_defect_5383_g001 - running 49/49 tests
Gremlin 48/75: user_defect_5383_g002 - running 49/49 tests
Gremlin 49/75: user_defect_5383_g003 - running 49/49 tests
Gremlin 50/75: user_defect_5383_g004 - running 49/49 tests
Gremlin 51/75: user_defect_5383_g005 - running 49/49 tests
Gremlin 52/75: user_defect_5383_g006 - running 49/49 tests
Gremlin 53/75: user_defect_5383_g007 - running 49/49 tests
Gremlin 54/75: user_defect_5383_g008 - running 49/49 tests
Gremlin 55/75: user_defect_5383_g009 - running 49/49 tests
Gremlin 56/75: user_defect_5383_g010 - running 49/49 tests
Gremlin 57/75: user_defect_5383_g011 - running 49/49 tests
Gremlin 58/75: user_defect_5383_g012 - running 49/49 tests
Gremlin 59/75: user_defect_5383_g013 - running 49/49 tests
Gremlin 60/75: user_defect_5383_g014 - running 49/49 tests
Gremlin 61/75: user_defect_5383_g015 - running 49/49 tests
Gremlin 62/75: user_defect_5383_g016 - running 49/49 tests
Gremlin 63/75: user_defect_5383_g017 - running 49/49 tests
Gremlin 64/75: user_defect_5383_g018 - running 49/49 tests
Gremlin 65/75: user_defect_5383_g019 - running 49/49 tests
Gremlin 66/75: user_defect_5383_g020 - running 49/49 tests
Gremlin 67/75: user_defect_5383_g021 - running 49/49 tests
Gremlin 68/75: user_defect_5383_g022 - running 49/49 tests
Gremlin 69/75: user_defect_5383_g023 - running 49/49 tests
Gremlin 70/75: user_defect_5383_g024 - running 49/49 tests
Gremlin 71/75: user_defect_5383_g025 - running 49/49 tests
Gremlin 72/75: user_defect_5383_g026 - running 49/49 tests
Gremlin 73/75: user_defect_5383_g027 - running 49/49 tests
Gremlin 74/75: user_defect_5383_g028 - running 49/49 tests
Gremlin 75/75: user_defect_5383_g029 - running 49/49 tests
```

```
=====
Zapped: 75 gremlins (100%)
Survived: 0 gremlins (0%)

Run with --gremlin-report-html for detailed report
```

Приложение 6. Тест с мутациями(гремлинами)

Вывод

Модуль `user.py` успешно прошел полный цикл тестирования. Все 49 тестов выполнены без ошибок, что подтверждает корректность реализации функций валидации email, пароля и возраста. Покрытие кода составило 96%, что превышает установленный целевой показатель в 90%, а покрытие ветвей достигло 92%, гарантируя проверку всех логических путей выполнения. Мутационное тестирование с использованием `pytest-gremlins` показало 100% эффективность — все 75 искусственно внесенных ошибок (мутаций) были обнаружены тестами. Найденные в ходе работы дефекты, включая отсутствие проверки букв в пароле, неполную валидацию email и пропуск проверки типа и границ возраста, были успешно исправлены. Таким образом, модуль полностью готов к интеграции в основную систему бронирования.